RELAZIONE TECNICA

NASA Asteroids Dashboard – Analisi Dati NeoWs

Chiara Polizzi

INDICE DEL DOCUMENTO

1. Introduzione e Obiettivi del Progetto

2. Scelte Architetturali e Stack Tecnologico

3. Progettazione del Database e Normalizzazione (Modello Logico)

3.1 Analisi Dati e Ottimizzazione tramite Viste (Livello DB)

3.2 Interazione col Database e Logica delle Query (Livello DAO)

4. Architettura Backend e Pattern MVC

5. Flussi di Esecuzione e Interazioni Client-Server

6. Ottimizzazione, Performance e Gestione della Concorrenza

7. Insight Ricavati dall'Analisi dei Dati

8. Riflessione Personale, Aneddoto e Sviluppi Futuri

EXECUTIVE SUMMARY

Questo documento descrive la progettazione e l'implementazione di una soluzione Full-Stack containerizzata per il recupero, la persistenza e l'analisi dei dati astronomici NASA NeoWs (Near Earth Objects). Il sistema è stato ingegnerizzato con un focus specifico su performance, sicurezza e integrità dei dati.

Highlight Architetturali e Scelte Chiave:

- **Backend (Node.js ed Express):** Architettura modulare basata sul pattern MVC. L'interazione con il database è stata incapsulata tramite pattern DAO (Data Access Object) per disaccoppiare la logica di business dalle query SQL.
- **Database (PostgreSQL):** Scelto per garantire una rigorosa integrità referenziale tramite chiavi esterne e vincoli ON DELETE CASCADE. Le query sfruttano comandi avanzati come UPSERT (ON CONFLICT DO UPDATE), Transazioni atomiche (BEGIN...COMMIT) e Viste SQL analitiche (VIEW) per delegare il carico computazionale al motore del DB.
- **Frontend (Vanilla JS e Chart.js):** Sviluppato interamente come Single Page Application (SPA) asincrona. Utilizza chiamate API in background (fetch) per aggiornare sezioni specifiche del DOM in tempo reale, implementando caricamenti paralleli e paginazione dinamica per ottimizzare l'esperienza utente.
- **Sicurezza e Concorrenza:** Prevenzione attiva delle vulnerabilità SQL Injection tramite Prepared Statements. Gestione sicura degli accessi concorrenti al database sfruttando la natura single-thread di Node.js combinata ai lock nativi di PostgreSQL per mantenere la coerenza logica.

1. Introduzione e Obiettivi del Progetto

Gli obiettivi principali del sistema sono:

1. **Integrazione e Persistenza:** Connessione all'API NASA per il download dei dati sugli asteroidi e salvataggio permanente in un database relazionale locale.
2. **Dashboard Interattiva:** Creazione di un'interfaccia utente asincrona che, tramite grafici e tabelle, metta in risalto informazioni chiave come la velocità di passaggio e il grado di pericolosità dei corpi celesti.
3. **Filtri Avanzati e Categorizzazione Spaziale:** Possibilità per l'utente di interrogare i dati storici in base a periodi temporali specifici (range di date per il grafico principale). Inoltre, per l'analisi geografica, inserendo un anno (e opzionalmente un mese di riferimento), il sistema elabora gli avvistamenti e li restituisce suddividendoli a schermo in base al corpo celeste orbitato (Terra, Marte o Giove).

Il tema degli asteroidi è stato scelto non solo per la sua rilevanza scientifica, ma perché l'API offre parametri numerici complessi (distanze, velocità, dati orbitali). Questi dati si prestano perfettamente per dimostrare solide competenze nella strutturazione di un database relazionale e nella stesura di query analitiche avanzate.

2. Scelte Architetturali e Stack Tecnologico

Per la realizzazione del progetto, ho scelto di evolvere il mio stack tecnologico superando l'approccio tradizionale basato su Java e Servlet (utilizzato nei miei passati progetti universitari), orientandomi verso soluzioni più moderne, autonome e adatte a un'architettura a container.

Ogni strumento è stato selezionato per rispondere a un'esigenza specifica:

1. **Backend (Node.js ed Express):** A differenza di Java, che necessita di un server ospite pesante (come Apache Tomcat) per eseguire l'applicativo web, Node.js permette di creare un server autonomo e molto leggero. Inoltre, i dati della NASA viaggiano in formato JSON: Node.js, essendo basato su JavaScript, li elabora in modo nativo e immediato, eliminando la necessità di utilizzare librerie di conversione esterne (come Gson o Jackson) che sarebbero invece obbligatorie in Java.
2. **Database (PostgreSQL):** Sebbene Node.js venga spesso abbinato a database non relazionali, ho scelto PostgreSQL per garantire una rigorosa integrità referenziale. Il progetto richiede infatti la gestione di relazioni complesse, come l'associazione di note personali a specifici asteroidi da parte di diversi profili utente. L'uso di un database relazionale previene in modo nativo e automatico la creazione di "dati orfani" o incongruenze strutturali.
   Ho preferito PostgreSQL a MySQL per due motivazioni: in primo luogo, PostgreSQL gestisce in modo più flessibile e performante le View analitiche complesse e le funzioni di estrazione

delle date; in secondo luogo, offre una gestione superiore della concorrenza tramite lock a livello di riga, una caratteristica fondamentale per garantire la coerenza dei dati qualora operazioni asincrone dovessero sovrapporsi (es. il reset di un profilo e il salvataggio simultaneo di una nota da parte dell'utente).

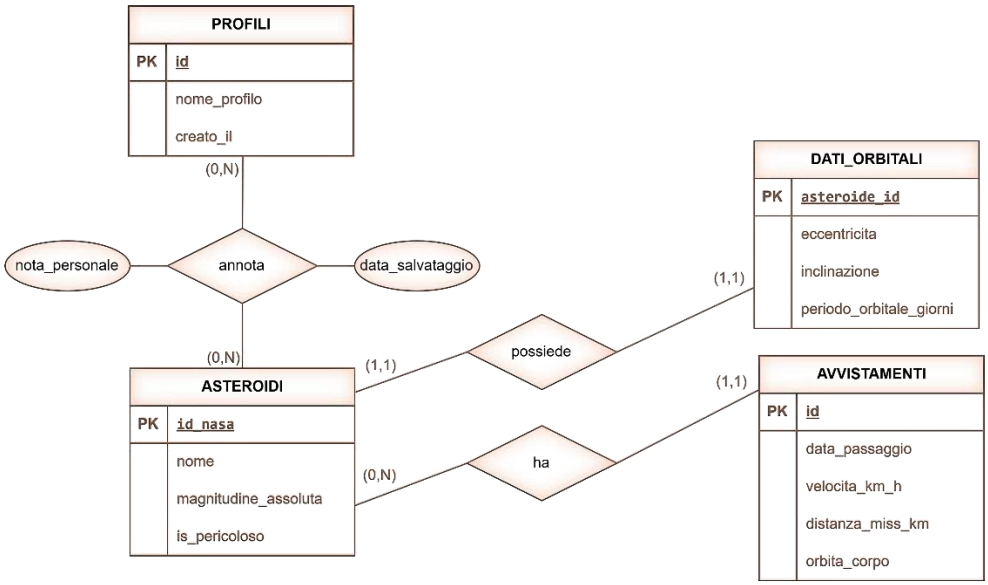
3. **Frontend (JavaScript Vanilla e Chart.js):** Per sviluppare la Single Page Application (SPA) non ho impiegato framework pesanti come React o Angular, sovradimensionati per questo contesto. Ho utilizzato JavaScript puro per gestire l'interazione sul DOM e le chiamate asincrone (Fetch API) verso il server. Questo approccio garantisce prestazioni elevate: l'interfaccia aggiorna solo i grafici e le liste necessarie, senza mai ricaricare l'intera pagina web.
4. **Infrastruttura (Docker e Volumi Persistenti):** L'intero sistema è stato "containerizzato" per renderlo immediatamente replicabile su qualsiasi macchina. L'utilizzo dei Volumi Docker assicura inoltre che i dati scaricati, le note scritte e i profili utente vengano salvati in modo permanente, sopravvivendo allo spegnimento o al riavvio dei container.

Queste scelte riflettono la precisa volontà di creare un'applicazione modulare e pulita (pattern Model-View-Controller), applicando le ottime pratiche di separazione delle responsabilità e robustezza del codice apprese durante la mia esperienza lavorativa e universitaria.

3. Progettazione del Database e Normalizzazione (Modello Logico)

Il database relazionale PostgreSQL è stato strutturato seguendo i principi della normalizzazione, con l'obiettivo di mantenere le interrogazioni leggere ed evitare la duplicazione dei dati. La progettazione si articola su strategie specifiche per ottimizzare sia lo spazio di archiviazione che i tempi di interrogazione dei dati:

- **Partizionamento Verticale (Relazione 1:1):** Le informazioni anagrafiche di base dei corpi celesti sono state isolate nella tabella principale asteroidi. I parametri fisici e matematici più complessi sono stati invece delegati alla tabella dati_orbitali tramite una relazione 1:1. Questa separazione alleggerisce le query principali: quando la dashboard deve solo compilare una lista di nomi, il database non è costretto a caricare inutilmente anche i dati matematici pesanti.
- **Gestione Annotazioni (Relazione N:M):** La funzionalità di salvataggio delle annotazioni è stata modellata come una relazione molti-a-molti tra le entità Profili e Asteroidi. A livello logico, questo si è tradotto nell'implementazione della tabella di congiunzione note_profili. L'utilizzo di chiavi esterne con vincoli ON DELETE CASCADE assicura la piena integrità referenziale del sistema, prevenendo in modo automatico la creazione di dati orfani (es. note associate a profili inesistenti o a corpi celesti non più tracciati).



Di seguito viene riportato lo script SQL, estratto dal file init.sql, utilizzato per la creazione delle tabelle nel database:

```
-- Tabella profili
CREATE TABLE IF NOT EXISTS profili (
   id SERIAL PRIMARY KEY,
   nome_profilo VARCHAR(50) NOT NULL,
   creato_il TIMESTAMP DEFAULT CURRENT_TIMESTAMP
   CONSTRAINT limite_profili CHECK (id <= 4)
);

-- Inserimento dei 4 profili di default
INSERT INTO profili (id, nome_profilo)
VALUES (1, 'Profilo 1'), (2, 'Profilo 2'), (3, 'Profilo 3'), (4, 'Profilo 4')
ON CONFLICT (id) DO NOTHING;

-- Tabella Anagrafica: Dati statici dell'asteroide
CREATE TABLE IF NOT EXISTS asteroidi (
   id_nasa VARCHAR(50) PRIMARY KEY,
   nome VARCHAR(100) NOT NULL,
   magnitudine_assoluta DOUBLE PRECISION,
   is_pericoloso BOOLEAN DEFAULT FALSE
);

-- Tabella Dati Orbitali
CREATE TABLE IF NOT EXISTS dati_orbitali (
   asteroide_id VARCHAR(50) PRIMARY KEY REFERENCES asteroidi(id_nasa) ON DELETE CASCADE,
   eccentricita DOUBLE PRECISION,
   inclinazione DOUBLE PRECISION,
   periodo_orbitale_giorni DOUBLE PRECISION
);
```

```
-- Tabella Avvistamenti: Dati variabili per ogni passaggio
CREATE TABLE IF NOT EXISTS avvistamenti (
   id SERIAL PRIMARY KEY,
   asteroide_id VARCHAR(50) REFERENCES asteroidi(id_nasa) ON DELETE CASCADE,
   data_passaggio DATE NOT NULL,
   velocita_km_h DOUBLE PRECISION,
   distanza_miss_km DOUBLE PRECISION,
   orbita_corpo VARCHAR(50),
   CONSTRAINT avvistamento_unico UNIQUE(asteroide_id, data_passaggio)
);

-- Tabella Preferiti E Note
CREATE TABLE IF NOT EXISTS note_profili (
   profilo_id INTEGER REFERENCES profili(id) ON DELETE CASCADE,
   asteroide_id VARCHAR(50) REFERENCES asteroidi(id_nasa) ON DELETE CASCADE,
   nota_personale TEXT,
   data_salvataggio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   PRIMARY KEY (profilo_id, asteroide_id)
);
```

3.1 Analisi Dati e Ottimizzazione tramite Viste (Livello DB)

Per mantenere il backend Node.js estremamente leggero ed evitare che calcoli costosi blocchino il suo unico thread, le operazioni computazionali e di filtraggio più pesanti sono state demandate direttamente al motore del database tramite l'utilizzo di Viste SQL analitiche.

1. **Vista Analitica del Rischio (v_insight_rischio_massimo):** Invece di calcolare le statistiche di pericolosità ciclando grandi array in JavaScript, ho creato una vista dedicata. Questa view unisce le tabelle anagrafiche e storiche (JOIN) e sfrutta le funzioni aggregate per estrarre istantaneamente la distanza minima (MIN) e la velocità massima (MAX) raggiunte nei vari passaggi.

2. **Vista per la Categorizzazione Spaziale (v_mappa_pianeti):** Progettata appositamente per alimentare la mappa geografica del frontend. La vista utilizza le funzioni native EXTRACT(MONTH FROM ...) ed EXTRACT(YEAR FROM ...) direttamente a livello di database. Questo approccio permette al backend di applicare filtri esatti sui mesi e sugli anni con query semplicissime, delegando al frontend la sola logica di smistamento visivo.

Di seguito viene riportato lo script SQL, estratto dal file init.sql, utilizzato per la generazione delle Viste:

```
-- VIEW 1: Indice di Rischio (Insight: unisce pericolosità, distanza minima e velocità)
CREATE OR REPLACE VIEW v_insight_rischio_massimo AS
SELECT
   a.id_nasa,
   a.nome,
   MIN(av.distanza_miss_km) AS distanza_piu_vicina_km,
```

```
    MAX(av.velocita_km_h) AS velocita_max_raggiunta,
    COUNT(av.id) AS numero_avvistamenti_totali
FROM asteroidi a
JOIN avvistamenti av ON a.id_nasa = av.asteroide_id
WHERE a.is_pericoloso = TRUE
GROUP BY a.id_nasa, a.nome
ORDER BY distanza_piu_vicina_km ASC
LIMIT 10;

-- VIEW 2: Dati per il disegno grafico dei pianeti (per il frontend)
CREATE OR REPLACE VIEW v_mappa_pianeti AS
SELECT
    a.id_nasa,
    a.nome,
    a.is_pericoloso,
    av.data_passaggio,
    EXTRACT(MONTH FROM av.data_passaggio) AS mese_passaggio,
    EXTRACT(YEAR FROM av.data_passaggio) AS anno_passaggio,
    av.distanza_miss_km,
    av.orbita_corpo
FROM asteroidi a
JOIN avvistamenti av ON a.id_nasa = av.asteroide_id;
```

3.2 Interazione col Database e Logica delle Query (Livello DAO)

Per separare nettamente la logica di persistenza dal resto dell'applicazione, ho implementato il pattern DAO (Data Access Object) all'interno della cartella /models. Tutte le query che prevedono un input utente utilizzano i Prepared Statements (i parametri con il simbolo $1, $2), una pratica di sicurezza fondamentale per gestire in modo sicuro i dati in ingresso e prevenire attacchi di tipo SQL Injection.

Di seguito si evidenziano le quattro query principali che definiscono il "motore" del progetto, pensate per gestire la concorrenza e garantire scalabilità:

1. **Sincronizzazione Idempotente (Caching NASA):**

   ```
   INSERT INTO asteroidi (id_nasa, nome, magnitudine_assoluta, is_pericoloso)
   VALUES ($1, $2, $3, $4)
   ON CONFLICT (id_nasa) DO NOTHING;
   ```

   La query di inserimento dei dati provenienti dall'API NASA è progettata per gestire chiamate ripetute in totale sicurezza. Se si tenta di salvare un asteroide già storicizzato, PostgreSQL intercetta la collisione delle chiavi primarie e ignora la singola istruzione. Questo permette aggiornamenti incrementali fluidi, ottimizza i tempi di elaborazione del database ed evita la cancellazione accidentale dei dati correlati (come le note degli utenti).

2. **Paginazione Server-Side Dinamica:**

   ```
   SELECT a.id_nasa, a.nome, v.velocita_km_h, v.data_passaggio
   FROM asteroidi a JOIN avvistamenti v ON a.id_nasa = v.asteroide_id
   WHERE v.data_passaggio >= $1 AND v.data_passaggio <= $2
   ORDER BY v.velocita_km_h DESC LIMIT $3 OFFSET $4;
   ```

   Invece di scaricare l'intero storico degli asteroidi, la query delega la scrematura al database. L'utilizzo dei comandi LIMIT e OFFSET garantisce che PostgreSQL invii al backend solo il blocco esatto di record richiesti (es. 10 alla volta). In questo modo i tempi di risposta dell'applicazione restano invariati e veloci anche con milioni di righe storicizzate.

3. **Operazioni "UPSERT" (Salvataggio Note):**

   ```
   INSERT INTO note_profili (profilo_id, asteroide_id, nota_personale)
   VALUES ($1, $2, $3)
   ON CONFLICT (profilo_id, asteroide_id)
   DO UPDATE SET nota_personale = EXCLUDED.nota_personale, data_salvataggio =
   CURRENT_TIMESTAMP;
   ```

   Per gestire la scrittura delle annotazioni, non ho utilizzato due query separate (SELECT di controllo seguita da INSERT/UPDATE), poiché avrebbe causato latenza di rete. L'uso della clausola ON CONFLICT DO UPDATE (tecnica Upsert) rende l'operazione sicura e risolutiva in un solo passaggio: se la nota non esiste viene creata, se esiste già viene aggiornato il testo.

4. **Transazioni e Integrità Logica (Reset del Profilo):**

   ```
   BEGIN;
   DELETE FROM note_profili WHERE profilo_id = $1;
   UPDATE profili SET nome_profilo = 'Profilo ' || $1 WHERE id = $1;
   COMMIT;
   ```

   La funzione di reset deve compiere due azioni distinte: cancellare le note e ripristinare il nome. Per evitare che un'eccezione a runtime esegua solo la prima operazione lasciando il database in uno stato incoerente, i comandi sono racchiusi in un blocco transazionale. L'atomicità garantisce che o tutte le modifiche vengono applicate (COMMIT), oppure, in caso di errore, annullate integralmente tramite ROLLBACK.

4. Architettura Backend e Pattern MVC

Per garantire scalabilità e pulizia del codice, il backend Node.js è stato refattorizzato seguendo il pattern architetturale Model-View-Controller (MVC). Questa organizzazione modulare disaccoppia le responsabilità, riprendendo l'approccio strutturato che ho utilizzato nella mia esperienza lavorativa e ancora prima universitaria.

L'applicativo è diviso nei seguenti moduli logici:

1. **Entry-Point (app.js):** Il file principale è stato snellito al massimo. Si occupa esclusivamente di avviare il server Express, iniettare i middleware necessari (come il parser JSON) e importare le rotte principali, delegando il resto della logica.
2. **Routes (/routes):** Fungono da "centralino" dell'applicazione. Intercettano le chiamate HTTP in ingresso e le smistano ai controller corretti, mantenendo l'elenco degli endpoint separato dalla logica esecutiva.
3. **Controllers (/controllers):** Rappresentano la logica di business. Il controller riceve la richiesta, esegue una validazione degli input (es. verifica dei limiti operativi per prevenire query anomale) e interroga i layer sottostanti.
4. **Models (/models):** Questo strato incapsula l'interazione diretta con PostgreSQL. Riprendendo il pattern DAO (Data Access Object) utilizzato nei miei progetti universitari in Java, i modelli nascondono le complessità SQL al resto dell'applicazione.
5. **Services (/services):** Moduli dedicati esclusivamente alla comunicazione con l'esterno. Il NasaApiService gestisce le chiamate asincrone tramite Axios verso gli endpoint pubblici, permettendo ai controller di rimanere puliti e focalizzati unicamente sulla logica applicativa interna.

5. Flussi di Esecuzione e Interazioni Client-Server

L'interfaccia utente è stata sviluppata seguendo l'approccio Single Page Application (SPA), quindi l'interazione dell'utente non causa mai il ricaricamento completo della pagina. Le chiamate di rete avvengono in modo asincrono (tramite Fetch API) e aggiornano istantaneamente solo porzioni specifiche del DOM (Document Object Model).

Per evitare ripetizioni, i flussi di esecuzione sono stati raggruppati nelle seguenti macrocategorie:

1. **Flusso di Sincronizzazione Dati (NASA API -> Database):**
    - **Azione:** L'utente richiede l'aggiornamento cliccando "Sincronizza Dati NASA". Il frontend invia una GET asincrona a /api/system/fetch-data.
    - **Elaborazione:** Il Service dedicato effettua una chiamata ad Axios verso l'API NeoWs. Il backend riceve il JSON, lo filtra snellendo il payload.
    - **Persistenza:** Viene eseguita una query di INSERT. Grazie all'istruzione ON CONFLICT DO NOTHING, il database implementa una logica di caching locale intelligente: previene duplicati senza generare errori nel caso di asteroidi già presenti, velocizzando l'operazione.
    - **Feedback:** Il server risponde e il frontend notifica l'utente tramite messaggi di stato, scatenando il refresh automatico e fluido del grafico e dell’insight sugli asteroidi più pericolosi.

2. **Flusso di Navigazione, Filtraggio e Paginazione del Grafico:**
    - **Azione:** L'utente inserisce un range di date o scorre le pagine. Viene inviata una richiesta al backend con i parametri di filtro (es. startDate, endDate, page).
    - **Elaborazione:** Il Model non carica l'intero dataset per il grafico, ma interroga PostgreSQL richiedendo i dati a blocchi (10 record alla volta) tramite le clausole LIMIT e OFFSET.
    - **Rendering Fluido:** Il frontend riceve il pacchetto JSON ridotto e aggiorna l'istanza di Chart.js. Invece di distruggere e ricreare il grafico (causando glitch visivi), i nuovi dati vengono iniettati dinamicamente, permettendo alla libreria di eseguire animazioni di transizione fluide ed eleganti.
3. **Flusso di Mappatura Spaziale e Insight Analitici:**
    - **Azione:** L'utente richiede l'analisi storica inserendo un anno (e opzionalmente un mese) scegliendo eventualmente tra quelli con più avvistamenti.
    - **Elaborazione:** Il backend interroga le Viste SQL pre-calcolate e viene restituita la lista degli avvistamenti.
    - **Mappa Spaziale:** Il frontend li smista visivamente nelle colonne corrispondenti all'orbita di passaggio (Terra, Marte, Giove).
4. **Flusso di Gestione Profili e Annotazioni (Operazioni CRUD):**
    - **Azione (Autocompilazione):** Per facilitare l'usabilità, cliccando su qualsiasi asteroide nel grafico o nelle liste, il frontend autocompila istantaneamente i moduli laterali con l'ID corretto, senza dover interrogare il server.
    - **Azione (Salvataggio):** L'utente invia una nota tramite una POST.
    - **Elaborazione:** Il Controller valida l'input e il Model esegue un'operazione idempotente (Upsert) sulla tabella note_profili.
    - **Feedback Immediato:** Ottenuto lo Status 200 (Success) dal server, il frontend re-interroga il database limitatamente alle note. Le operazioni avvengono istantaneamente, mantenendo la continuità di navigazione per l'utente.

6. Ottimizzazione, Performance e Gestione della Concorrenza

Il sistema è stato ingegnerizzato per gestire grandi moli di dati e interazioni simultanee garantendo stabilità visiva e integrità dei dati.

1. **Paginazione Server-Side:** Per mantenere il sistema reattivo indipendentemente dal volume di dati storicizzati dalla NASA, il filtraggio non è delegato al client. L'utilizzo di LIMIT e OFFSET assicura che il server e il browser processino unicamente i record necessari alla

singola visualizzazione (es. 10 asteroidi alla volta), garantendo un consumo di memoria costante e trasferimenti di rete ultrarapidi.

2. **Rendering Parallelo:** Il caricamento iniziale dei dati (profili, statistiche, grafico) avviene in parallelo per abbattere i tempi di attesa. Per evitare che l'inserimento simultaneo delle statistiche deformasse il layout interrompendo l'animazione di Chart.js, è stata assegnata un'altezza fissa al contenitore del grafico. Inoltre, il grafico viene inizializzato con valori a zero e immediatamente aggiornato con i dati reali; questo forza l'engine grafico a eseguire un'animazione senza bloccare il thread principale del browser.

3. **Gestione della Concorrenza e Single-Thread:** In fase di progettazione è stato analizzato lo scenario limite in cui vengano inviate contemporaneamente una richiesta di "Salvataggio Nota" e una di "Reset Profilo". Poiché Node.js elabora le richieste su un singolo thread, le operazioni vengono accodate e mai eseguite nel medesimo istante fisico (come avverrebbe in C++ o Java).

4. **Integrità Transazionale:** Anche nel caso in cui la scrittura della nota venga processata immediatamente dopo un reset (rischiando un'incoerenza logica), l'integrità fisica dei dati è blindata. Le operazioni critiche sul database sono racchiuse in Transazioni SQL (BEGIN...COMMIT) che garantiscono l'atomicità, e i lock nativi a livello di riga di PostgreSQL impediscono la collisione delle query in fase di scrittura.

7. Insight Ricavati dall'Analisi dei Dati

L'obiettivo della dashboard non è la semplice esposizione di dati grezzi, ma l'estrazione di insight significativi. Interrogando il database e analizzando i risultati restituiti dalle Viste SQL (VIEW), sono emersi pattern e correlazioni interessanti riguardo i Near Earth Objects:

1. **La Distanza non implica Pericolosità:** Incrociando i dati nella vista v_insight_rischio_massimo, è emerso un insight controintuitivo: la distanza minima di approccio (distanza_miss_km) non è un parametro sufficiente a determinare il flag is_pericoloso = TRUE fornita dall'API. Molti asteroidi nel database registrano passaggi estremamente ravvicinati all'orbita terrestre, ma sono classificati come innocui. Questo evidenzia che il modello NASA valuta il rischio combinando la prossimità orbitale con la magnitudine assoluta (le dimensioni stimate dell'oggetto).

2. **Distribuzione degli Avvistamenti e Frequenza Orbitale:**

   Tramite le query aggregate sui raggruppamenti per anno, si nota come la percentuale di asteroidi classificati come "pericolosi" sia statisticamente molto bassa rispetto al volume totale di Near Earth Objects tracciati. La maggior parte degli oggetti nel database locale presenta periodi orbitali ricorrenti ma orbite stabili.

8. Riflessione Personale, Aneddoto e Sviluppi Futuri

La progettazione di questo sistema è stata fortemente influenzata dalle mie esperienze professionali pregresse come Verification & Validation Engineer e Software Engineer presso TXT e-solutions.

Lavorando su sistemi software critici scritti in C++ e Ada95 , e occupandomi della gestione di accessi concorrenti al database e della correzione di bug complessi, ho appreso un principio fondamentale: non fidarsi mai ciecamente degli input esterni. Questa forma mentis mi ha guidato nella realizzazione di questo progetto, spingendomi a non considerare mai un dato "sicuro" solo perché proveniente da un'interfaccia controllata.

Questo approccio si è tradotto in una solida architettura difensiva e in diverse proposte per lo sviluppo futuro dell'applicativo:

1. **Validazione a Doppio Strato:** Attualmente, i dati in ingresso subiscono una validazione sia sul frontend sia sul backend. Ad esempio, il Controller impone dei limiti numerici massimi sulle query per prevenire tentativi di Denial of Service (DoS). Il backend agisce sempre in un ambiente "zero-trust".

2. **Refactoring e Validazione Centralizzata (Sviluppo Futuro):** In un'ottica prettamente Enterprise e di scalabilità, il prossimo passo prevede l'estrazione delle logiche di controllo dai Controller per demandarle a middleware di validazione dedicati, sfruttando librerie consolidate nell'ecosistema Node.js. Questo approccio replicherebbe fedelmente l'uso dei DTO (Data Transfer Object) tipico degli ambienti aziendali complessi come quello di TXT.

3. **Centralizzazione delle Costanti Operative (Sviluppo Futuro):** Per garantire un codice auto-documentante e una manutenzione agevole, i limiti operativi e i vincoli stringenti verrebbero spostati dai Controller a un file di configurazione globale (es. config/constants.js). In questo modo, qualsiasi modifica alle regole di business avverrebbe in un unico punto centralizzato.

4. **Optimistic Locking per la Concorrenza (Sviluppo Futuro):** Per risolvere definitivamente i casi limite di concorrenza, come lo scenario in cui l'accodamento asincrono di Node.js processi un "Salvataggio Nota" un istante dopo un "Reset Profilo", propongo l'introduzione di un campo version nella tabella dei profili. Se una richiesta in ritardo tentasse di scrivere su una versione obsoleta, verrebbe bloccata dal backend, garantendo una coerenza logica assoluta dei dati.

5. **Data Enrichment (Sviluppo Futuro):** L'API pubblica NeoWs della NASA è priva di contenuti multimediali o descrittivi, per migliorare l'esperienza utente e rendere l'applicazione distintiva il backend potrebbe eseguire una chiamata asincrona secondaria verso un'altra fonte dati (come la Wikipedia REST API o la NASA Image and Video Library), permettendo di incrociare i dati numerici con informazioni testuali e immagini reali.